

▼

实战篇

- 1 PaiGLI学习路线

更新时间: 2026年06月03日
- 2 ReAct和tool-call

更新时间: 2026年04月19日
- 3 plan和DAG

更新时间: 2026年04月21日
- 4 Memory系统

更新时间: 2026年04月21日
- 5 Multi-Agent

更新时间: 2026年04月23日
- 6 RAG代码检索

更新时间: 2026年04月23日
- 7 人工审批

更新时间: 2026年04月24日
- 8 多并发

更新时间: 2026年04月25日
- 9 联网搜索

更新时间: 2026年04月28日
- 10 接入MGP

更新时间: 2026年04月29日
- 11 DevTools-MGP

更新时间: 2026年05月06日
- 12 GDP-会话复用

更新时间: 2026年05月08日
- 13 web-aceess-Skill

更新时间: 2026年05月08日
- 14 多模态

更新时间: 2026年05月11日
- 15 微信里爽用PaiGLI

更新时间: 2026年06月13日

➤

简历篇

➤

面试篇

➤

补充篇

目录



Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

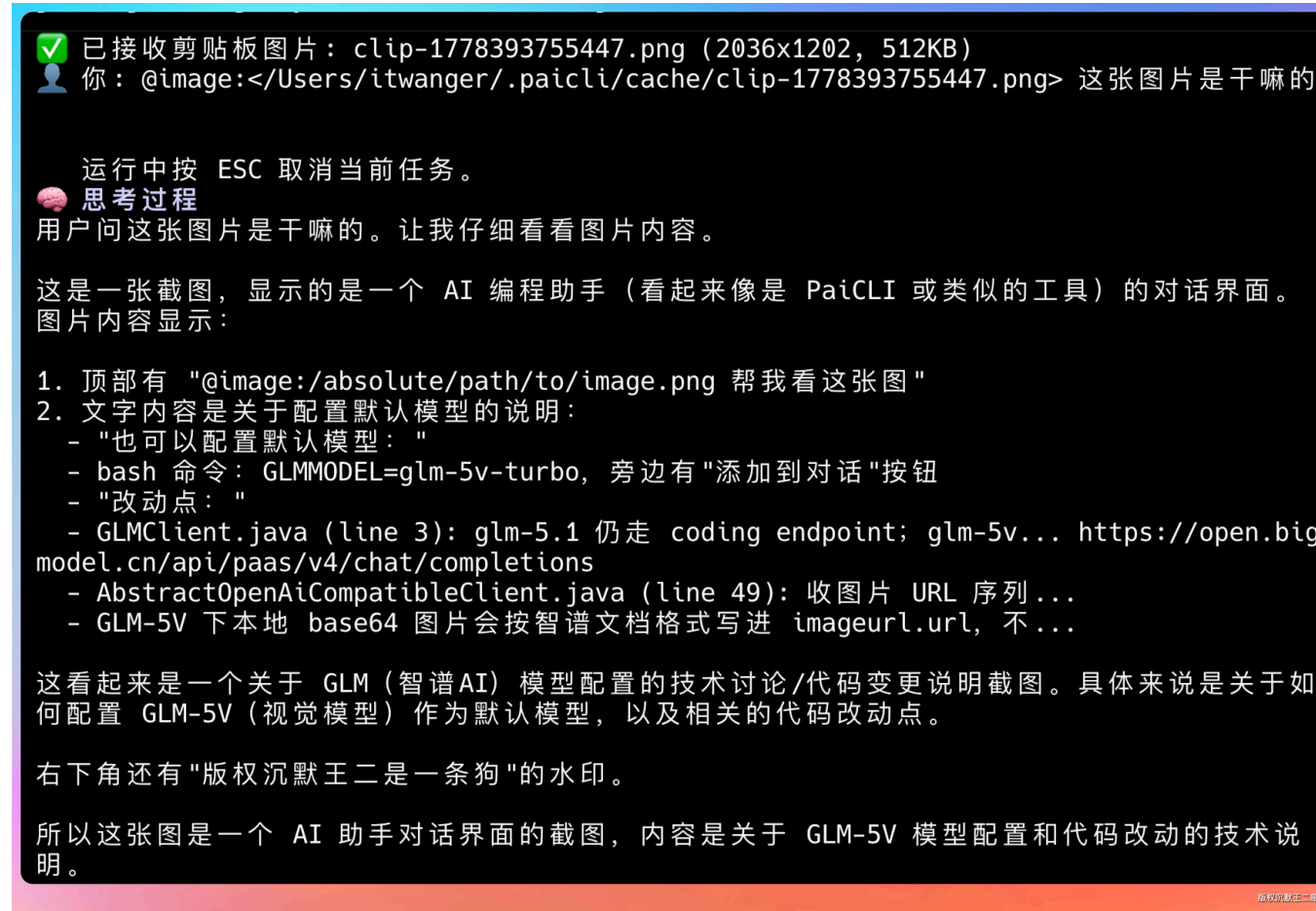
Agent 终于能看图了！ GLM-5V 让 PaiCLI 在图像识别上有了一双眼睛。

大家好，我是二哥呀。

PaiCLI 已经非常强大了，有 ReAct、Multi-Agent、MCP、Skill、Function Calling，基本上 Claude Code 有的功能都覆盖到了。

今天这篇，我们给 PaiCLI 再追加一个能力：图片输入。听起来简单，但真正做起来涉及到的东西很多。

这个功能的核心前提是多模态模型。单纯的文本模型是看不了图的，比如说 GLM-5.1，于是我们追加了 GLM-5V 模型的 endpoint。



来看看效果，把技术派的首页复制粘贴进去。



能准确识别出这些信息。

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

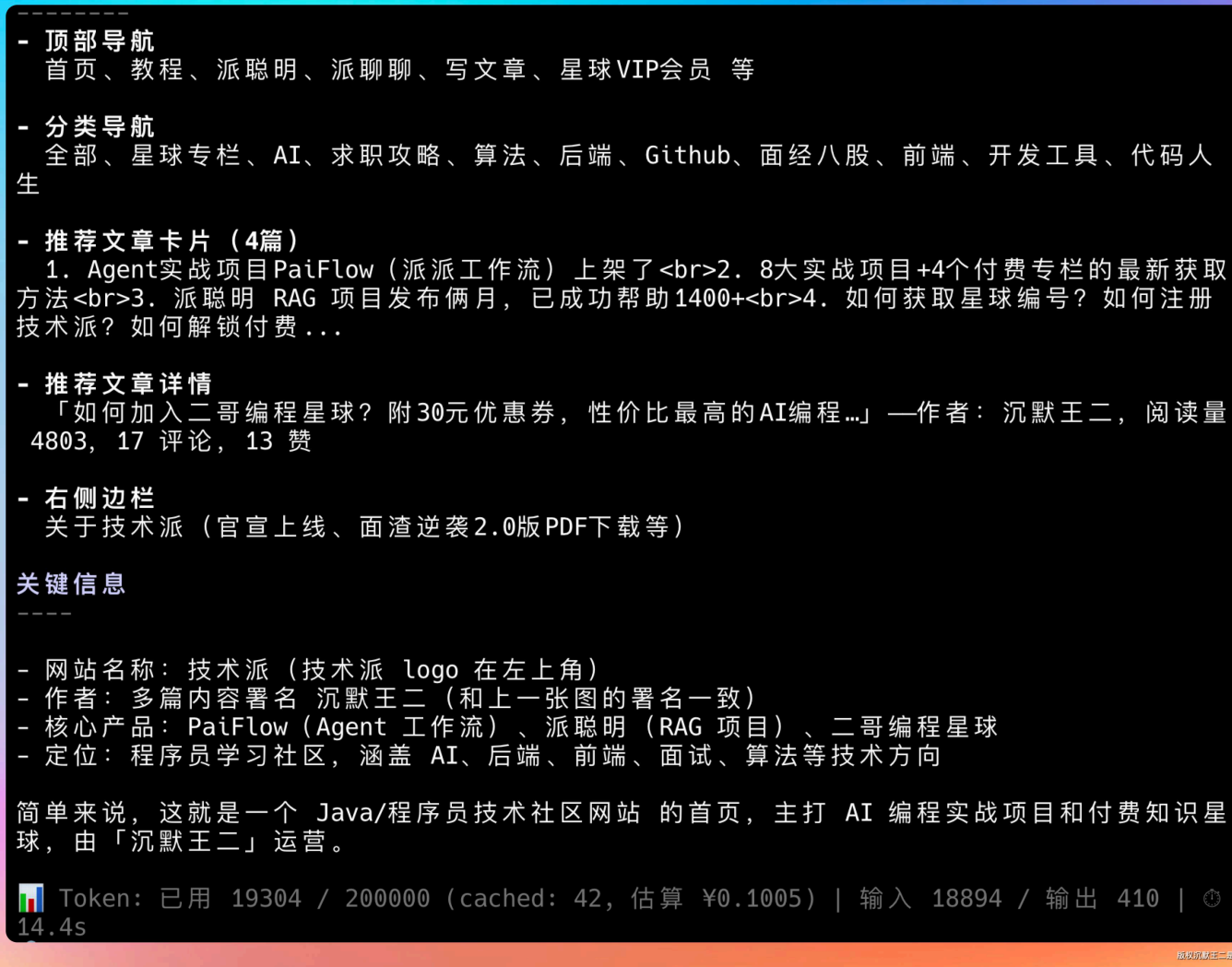
04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

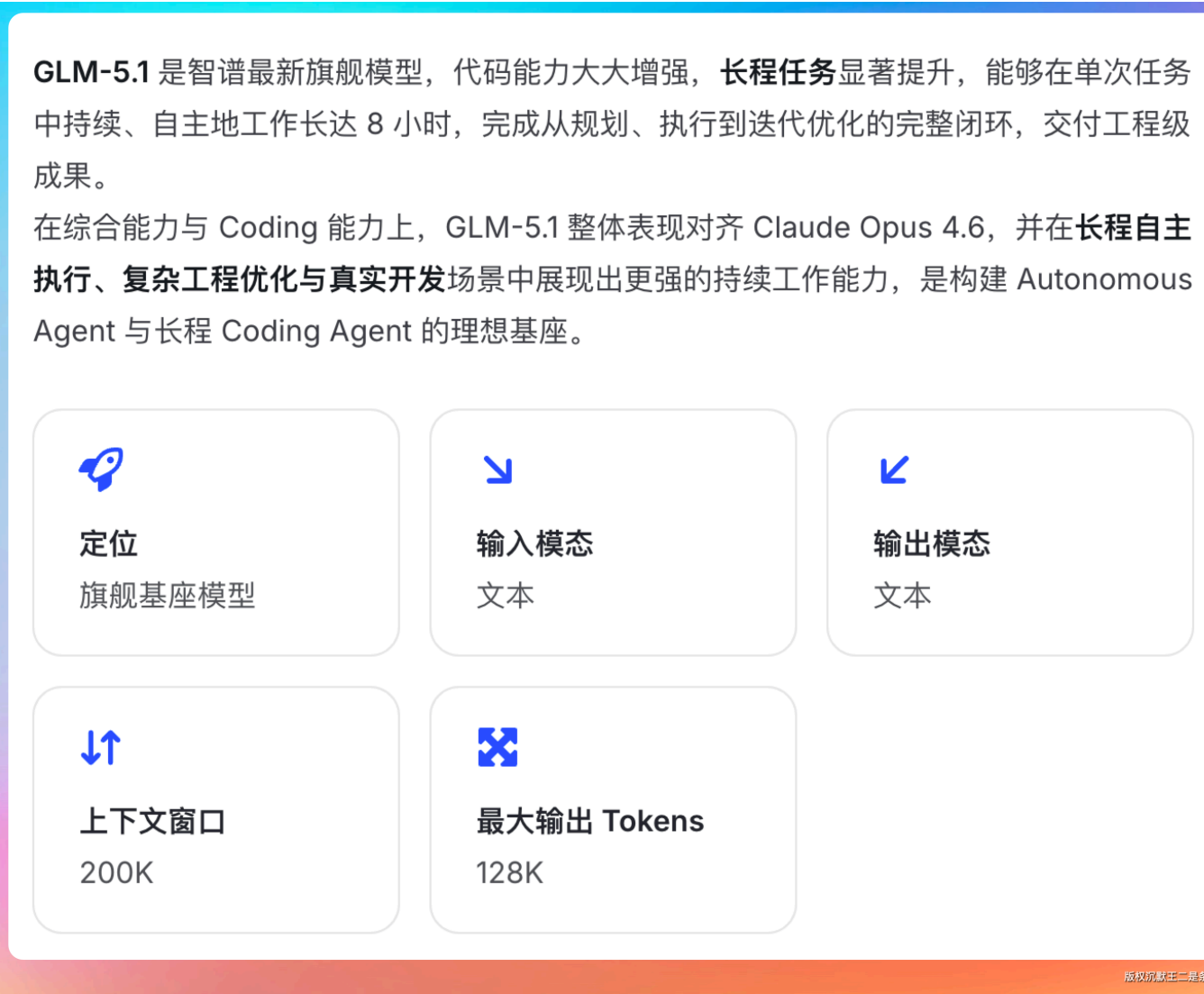


- 网站名称：技术派（技术派 logo 在左上角）
- 作者：多篇内容署名 沉默王二（和上一张图的署名一致）
- 核心产品：PaiFlow（Agent 工作流）、派聪明（RAG 项目）

01、为什么 GLM-5.1 看不了图

GLM-5.1 是一个纯文本大语言模型。它的输入只能是文本。

文本经过 Tokenizer 切成 token 序列，送进 Transformer 做注意力计算，输出也是 token 序列再解码回文本。整个推理过程中，模型的“感官”只有一个，就是文本。



GLM-5V 多了一个关键组件：Vision Encoder。

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

☆

定位

多模态 Coding 基座

↘

输入模态

视频、图像、文本、文件

↙

输出模态

文本

↕

上下文窗口

200K

⌘

最大输出 Tokens

128K

版权原主二是

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

这个 Vision Encoder 通常是一个预训练好的 ViT（Vision Transformer），它的工作是把一张图片转换成一组“视觉 token”。

1

Query-Key-Value 结构 (查询-键-值结构)

输入序列: “我 喜欢 深度 学习”

我

喜欢

深度

学习

Query (查询): 我 → [0.2, 0.5, 0.8, 0.3]

Key (键): 喜欢 → [0.4, 0.7, 0.2, 0.9]

Value (值): 深度 → [0.1, 0.3, 0.6, 0.4]

学习 → [0.8, 0.2, 0.5, 0.7]

Query (查询)

Key (键)

Value (值)

2

注意力分数计算 (Attention Score Calculation)

计算步骤:

1 Step 1: $Query \times Key^T$

$[0.2, 0.5, 0.8, 0.3] \times [0.4, 0.7, 0.2, 0.9]^T$

$= 0.2 \times 0.4 + 0.5 \times 0.7 + 0.8 \times 0.2 + 0.3 \times 0.9$

$= 0.08 + 0.35 + 0.16 + 0.27$

$= 0.86$

2 Step 2: Scale (除以 $\sqrt{d_k}$)

$0.86 / \sqrt{4} = 0.43$

3 Step 3: Softmax

$\exp(0.43) / \sum \exp = 0.35$

注意力分数: 0.35

(表示“我”对“喜欢”的关注程度)

3

加权求和可视化 (Weighted Sum Visualization)

注意力权重可视化:

我

喜欢

深度

学习

0.15

0.35

0.25

0.25

输出 = $0.15 \times Value(我) + 0.35 \times Value(喜欢) + 0.25 \times Value(深度) + 0.25 \times Value(学习)$

$= \sum_i \alpha_i \times Value(i)$

其中 α_i 是“我”对第 i 个词的注意力权重

4

自注意力 vs 交叉注意力 (Self-Attention vs Cross-Attention)

Self-Attention (自注意力)

Q, K, V 来自同一输入序列

输入: 一句话

“我喜欢学习”

我

喜欢

学习

用途: 理解句子内部词语关系

示例: 情感分析、文本分类等

Cross-Attention (交叉注意力)

Q 来自解码器, K, V 来自编码器

输入: 两句话

源: “I like study”

目标: “我喜欢学习”

编码器 (Encoder)

解码器 (Decoder)

用途: 机器翻译、文本对齐、问答系统等

5

多头注意力 (Multi-Head Attention)

8个注意力头并行计算:

Head 1: 关注语法结构

Head 2: 关注语义关系

Head 3: 关注位置信息

Head 4: 关注指代消解

Head 8: 关注长距离依赖

Concat (拼接) + Linear (线性变换) → 最终输出

每个头从不同角度捕获序列信息, 互补增强模型表达能力

6

完整公式 (Complete Formula)

$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

其中:

Q: Query matrix ($n_q \times d_k$)

K: Key matrix ($n_k \times d_k$)

V: Value matrix ($n_k \times d_v$)

d_k : Key 的维度

$\sqrt{d_k}$: 缩放因子防止点积过大

Q

n_q

d_k

$\times$

K^T

d_k

$\Rightarrow$

V

n_k

d_v

$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$

$\rightarrow$

n_q

d_v

注意力机制的优势

并行计算

长距离依赖

可解释性

灵活性

不像RNN需要顺序处理, 大幅提升训练效率

任意两个位置直接关联, 更好捕获长距离依赖关系

注意力权重可视化, 清晰展示模型关注点

适用于各种序列任务, 如NLP、CV、语音等

具体流程是这样的：

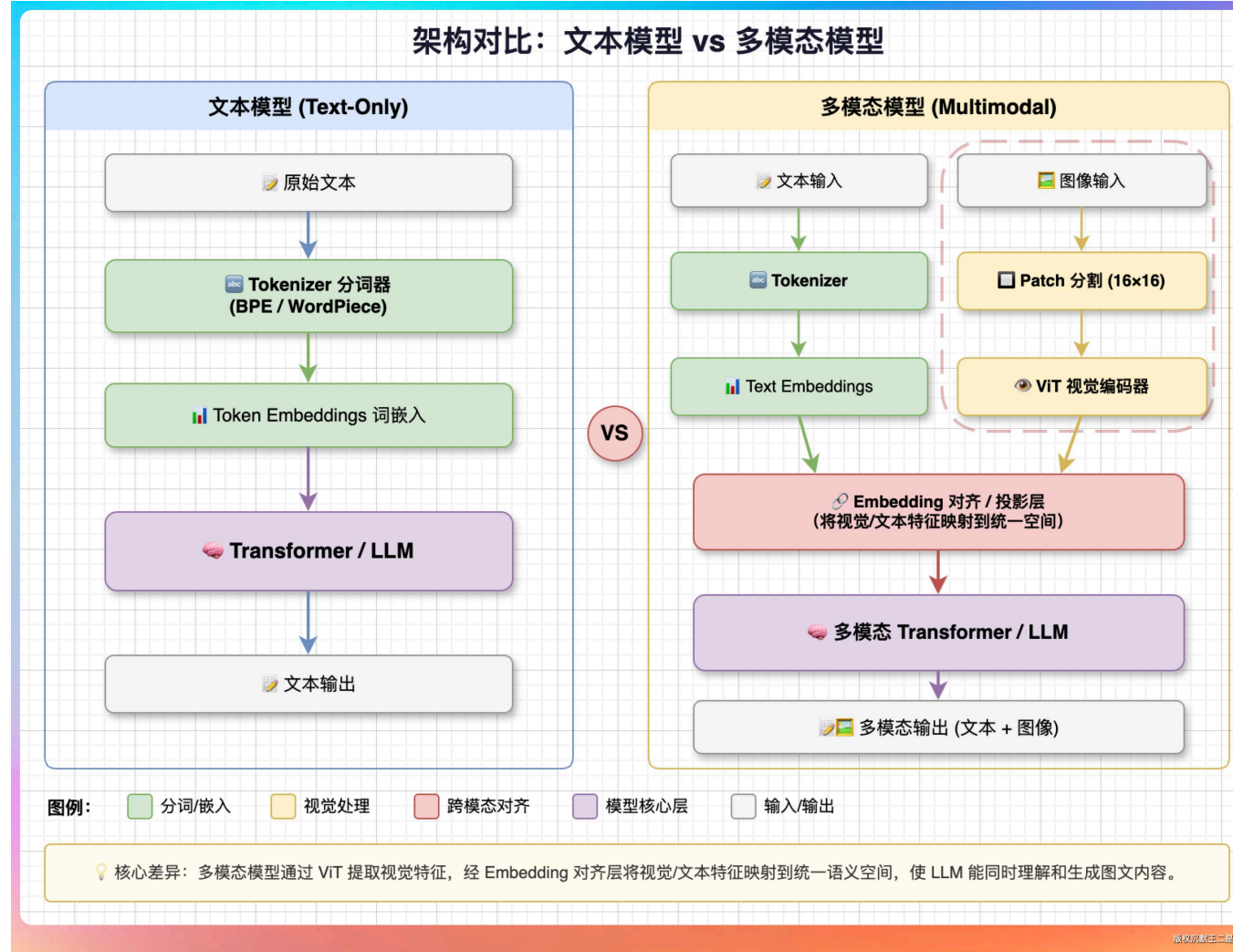
第一步，把图片切成固定大小的 patch（通常是 14x14 或 16x16 像素一个 patch）。一张 224x224 的图会被切成 16x16=256 个 patch。实际的大模型处理的图片分辨率更高，比如 2000x2000 的图会被切成几千个 patch。

第二步，每个 patch 经过 ViT 的线性投影层和多层 Transformer，变成一个固定维度的 embedding 向量。这些向量就是“视觉 token”，和文本 token 的 embedding 维度对齐。

https://paicoding.com/multimodal-paismart

4/13

第三步，可以粗略理解为视觉 token 和文本 token 进入同一个多模态推理过程。模型通过注意力机制，让文本 token 能“看到”视觉 token，视觉 token 也能参考文本上下文。官方更偏向把 GLM-5V 这套架构描述成“原生多模态融合”。



02、ContentPart 协议升级

搞清楚了多模态的原理，接下来看 PaiCLI 怎么在代码层面支持它。

第一步是改造 LLM 通信协议。

之前 PaiCLI 的 `LlmClient.Message` 里 `content` 就是一个 String，纯文本。Agent 发消息给模型，就是把字符串塞进 JSON 的 `content` 字段，模型返回的也是一个字符串。

多模态 Vision API 要求 `content` 不能是字符串，而是一个数组，里面可以混排 text block 和 image block，每个 block 有自己的 type 和数据。

实际发给 API 的 JSON 结构长这样：

json 复制代码

```
{
  "role": "user",
  "content": [
    {"type": "text", "text": "帮我分析下这张截图"},
    {"type": "image_url", "image_url": {"url": "data:image/png;base64,iVBORw0KGgoAAAANSUh..."}}
  ]
}
```

这意味着我们需要把 Message 的数据结构从“一个字符串”升级为“一个内容块列表”。

所以我们的策略是：给 Message 加一个 `contentParts` 字段，和原来的 `content` 字段并存。普通文本消息继续走 `content`，只有带图片的消息才用 `contentParts`：

java 复制代码

```
record Message(String role, String content, String reasoningContent,
               List<ToolCall> toolCalls, String toolCallId,
               List<ContentPart> contentParts) {

    public static Message user(List<ContentPart> contentParts) {
        return new Message("user", plainText(contentParts), null, null, null,
                           contentParts == null ? null : List.copyOf(contentParts));
    }

    public boolean hasContentParts() {
        return contentParts != null && !contentParts.isEmpty();
    }
}
```

注意 `Message.user()` 工厂方法里的 `plainText(contentParts)`，它会把 `contentParts` 里所有文本块拼接成一个纯文本 fallback 存到 `content` 字段。这样即使后续序列化逻辑走到不支持 `content array` 的分支，消息也不会丢失文本信息。

序列化的时候，`AbstractOpenAiCompatibleClient.appendMessageContent()` 会根据 `hasContentParts()` 决定走哪条路：

java 复制代码

```
private void appendMessageContent(ObjectNode msgNode, Message msg) {
    if (!msg.hasContentParts()) {
        msgNode.put("content", msg.content()); // 纯文本：直接塞字符串
        return;
    }
    ArrayNode contentArray = msgNode.putArray("content"); // 多模态：构建数组
    for (ContentPart part : msg.contentParts()) {
        if (part.isText()) {
            // text block
        } else if (part.isImage()) {
            // image_url block, 具体格式由子类 toImageUrl() 决定
        }
    }
}
```

base64 图片默认会被转成 `data:image/png;base64,<payload>` 格式的 data URI，塞进 `image_url.url` 字段。

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

03、@image 引用解析

用法很简单：`@image:./shot.png`。

less 复制代码

帮我分析下这张截图 @image:./shot.png

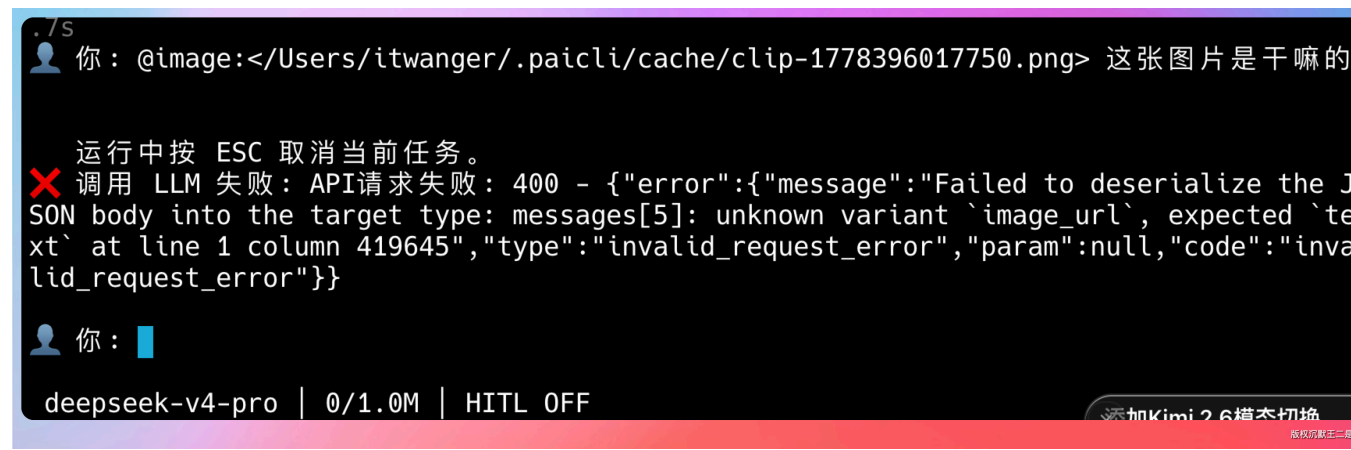
PaiCLI 会在终端里显示一条提示：`[已附加图片： ./shot.png, mimeType=image/png, bytes=...]`，然后模型的回复就是基于图片内容的分析了。

路径写法支持好几种：

bash 复制代码

@image:./shot.png # 相对路径
@image:/Users/itwanger/Desktop/error-log.png # 绝对路径
@image:file:///Users/itwanger/Desktop/shot.png # file:// 协议
@image:<file:///Users/itwanger/Desktop/path with spaces.png> # 尖括号包裹
@image:</Users/itwanger/Desktop/中文截图.png> # 中文路径

当然了，如果是纯文本模型比如 DeepSeek V4，是不支持的。



核心的正则表达式长这样：

java 复制代码

private static final Pattern IMAGE_REF = Pattern.compile(
 "@image:(<[^\>]+>|[\^\\s<>\\u2010-\\u206F\\u3000-\\u303F\\uFF00-\\uFFEF]+)"
 + " |@clipboard(?:[\\p{L}\\p{N}]_)");

尖括号包裹语法 `@image:<...>` 则是为了处理路径里有空格或特殊字符的情况。尖括号内可以放任何字符，直到遇到 `>` 才停止匹配。

解析出路径后，`ImageReferenceParser` 还会处理 `file://` 协议前缀和 percent-encoding。

04、@clipboard 剪贴板抓图

PaiCLI 支持两种剪贴板输入方式。

第一种是在对话里打 `@clipboard:`

CSS 复制代码

帮我看看这张图 @clipboard

第二种是直接按 `Ctrl+V`，PaiCLI 会自动抓取剪贴板图片，并在输入行末尾追加一个 `@image:` 引用。

PaiCLI 在 macOS 上走的是 AppleScript + osascript：

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

```
on run argv
  set outputPath to item 1 of argv
  try
    set pngData to (the clipboard as «class PNGf»)
  on error errMsg
    error "剪贴板里没有 PNG 数据"
  end try
  set fh to open for access (POSIX file outputPath as string) with write permission
  try
    set eof of fh to 0
    write pngData to fh
    close access fh
  on error errMsg
    try
      close access fh
    end try
    error errMsg
  end try
end run
```

«class PNGf» 是 macOS 剪贴板里 PNG 数据的 Apple Event 类型。截图工具放进剪贴板的就是这个格式。

但有些应用比如 Preview 和部分 Office 软件，往剪贴板里放的是 TIFF。所以 PaiCLI 有个兜底：PNG 抓不到就试 «class TIFF»，抓到 TIFF 后用系统自带的 /usr/bin/sips 转成 PNG。

Java 侧通过 `ProcessBuilder` 调用 `osascript`，脚本从 `stdin` 传入（不落临时文件），8 秒超时保护。整个冷启动大概 30ms，用户几乎无感知。

java 复制代码

```
Process process = new ProcessBuilder("/usr/bin/osascript", "-", outputPath).start();
try (var stdin = process.getOutputStream()) {
    stdin.write(script.getBytes(StandardCharsets.UTF_8));
}
boolean completed = process.waitFor(8, TimeUnit.SECONDS);
```

Linux 和 Windows 走的是 AWT 的标准 `Clipboard.getData(DataFlavor.imageFlavor)`，headless 环境（SSH / Docker）会直接提示“当前环境无 GUI”。

为什么不统一用 Java AWT 的 Clipboard API？

因为 macOS 的 Java AWT 实现有个已知问题：`DataFlavor.imageFlavor` 拿到的是 `BufferedImage` 对象，但这个对象在序列化回 PNG 字节数组的时候，颜色空间会从 Display P3 退化成 sRGB，某些截图的颜色会出现明显偏差。走 AppleScript 直接拿 PNG 原始字节就不存在这个问题，数据不经过 Java 的颜色空间转换，像素级保真。

05、MCP 截图

图片输入真正发挥威力的场景，是和 Chrome DevTools MCP 配合使用。

之前我们给 PaiCLI 接上了 Chrome DevTools MCP，Agent 能控制浏览器、导航页面、截图。但截图拿回来只有一段占位文字，模型看不到图片内容。

bash 复制代码

```
打开 https://www.apple.com 然后截图，告诉我首页主视觉里有什么
```

目录

Agent 终于能看图了！GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

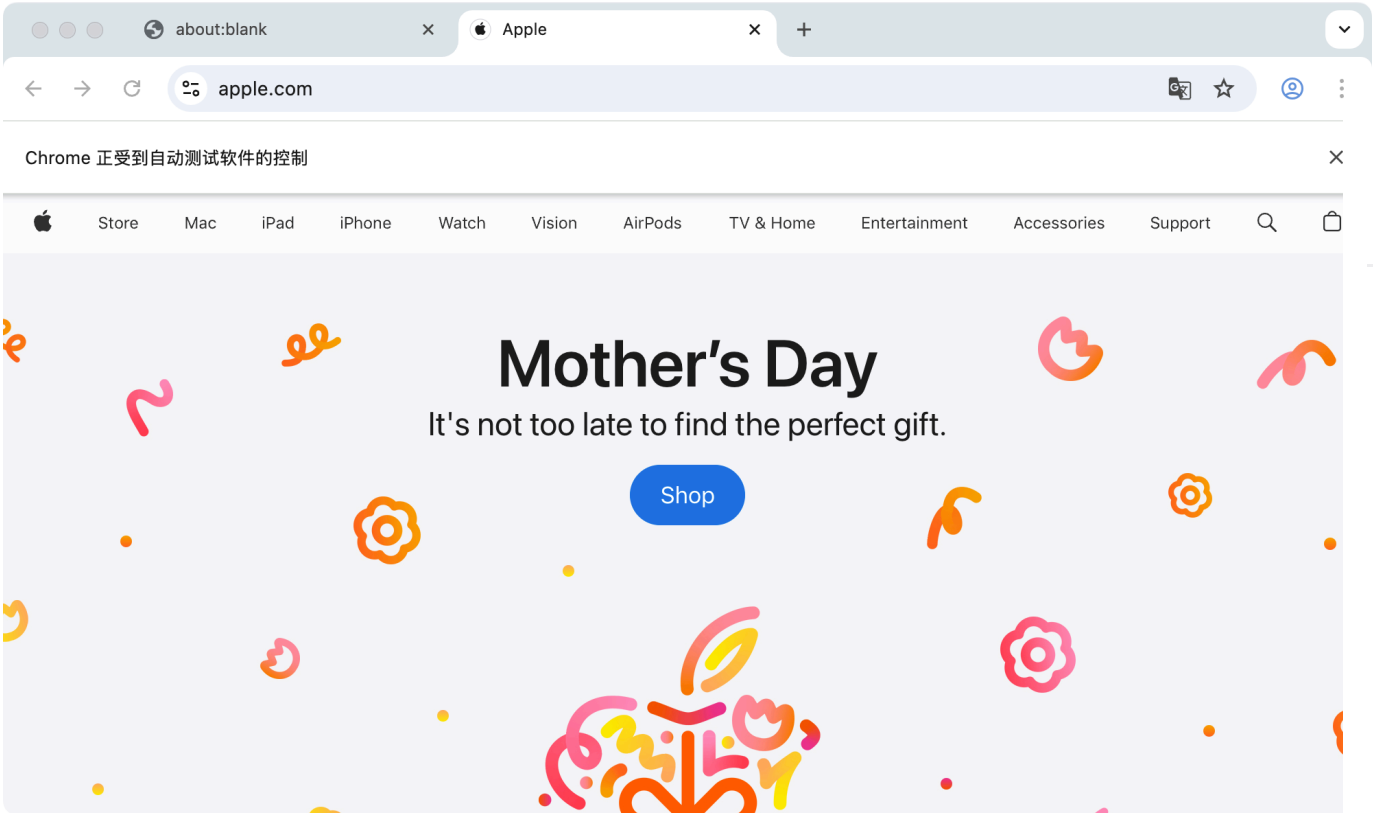
04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上



这个链路的实现涉及三个组件的协作。

首先是 `McpCallToolResult`。

MCP 工具返回的结果是一个 JSON 数组，里面的 content item 有 `type: "text"` 和 `type: "image"` 两种。PaiCLI 在 `toToolOutput()` 方法里会遍历这个数组，遇到 image 类型的 content，提取 base64 数据，经过 `ImageProcessor` 预处理后，存入 `ToolOutput` 的 `imageParts` 列表：

```
java 复制代码
if ("image".equals(type)) {
    ImageProcessor.ProcessedImage processed = ImageProcessor.fromBase64(item.data(),
    mimeType);
    imageParts.add(ImageProcessor.toContentPart(processed));
    // 同时生成文本 fallback，保证工具调用协议不被破坏
}
```

文本 fallback 也会同时生成，告诉模型“PaiCLI 会在下一轮把图片作为图片附件附加”。这样即使 provider 不支持图片输入，工具结果仍然是一条合法的 tool message。

然后是 `Agent.appendImageToolMessages()`。这个方法在工具执行完毕后被调用，检查每个工具结果里有没有 `imageParts`。如果有，就构造一条新的 user message，把图片作为 `ContentPart` 追加到对话历史里：

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

java 复制代码

```
private void appendImageToolMessages(List<ToolExecutionResult> toolResults) {
    for (ToolExecutionResult result : toolResults) {
        if (!result.hasImageParts()) {
            continue;
        }
        List<LlmClient.ContentPart> parts = new ArrayList<>();
        parts.add(LlmClient.ContentPart.text(
            "工具 " + result.name() + " 返回了图片内容，请结合上面的工具文本结果分析。"));
        parts.addAll(result.imageParts());
        conversationHistory.add(LlmClient.Message.user(parts));
    }
}
```

为什么用 user message 而不是直接把图片塞进 tool message?

因为 OpenAI API 规范里，tool role 的 message 只支持纯文本 content，不支持 content array。如果我们强行往 tool message 里塞图片 block，API 会返回 400 错误。

所以 PaiCLI 的处理是：tool message 放文本 fallback（告诉模型这个工具返回了图片），紧接着追加一条 user message 放真图。消息顺序变成 assistant(tool_calls) → tool(text fallback) → user(text + image block)。

还有一个容易忽略的问题：上下文膨胀。

每张图片经过 base64 编码后平均在 200KB~2MB，如果 Agent 执行了十几轮 ReAct 循环，每轮都带着历史图片，上下文会迅速爆掉。

PaiCLI 的解法是 pruneHistoricalImagePayloads()：在每一轮新的 ReAct 推理开始前，扫描对话历史中所有消息，把已经处理过的图片 block 替换成一行文本占位符【图片已省略，参见上文描述】。

只保留最近一轮的图片实体数据，更早的全部裁剪掉。这样模型仍然知道“之前看过什么图”，但不需要在每轮推理中重新消耗那些图片的 token，上下文窗口的利用效率大幅提升。

06、对比两张截图

多图输入是我最喜欢的一个功能。一条消息里可以同时附加多张图片：

less 复制代码

```
对比一下这两张截图差异 @image:./before.png @image:./after.png
```

PaiCLI 会分别处理两张图片，终端里会显示两条附加提示，模型能同时看到两张图并给出对比分析。

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

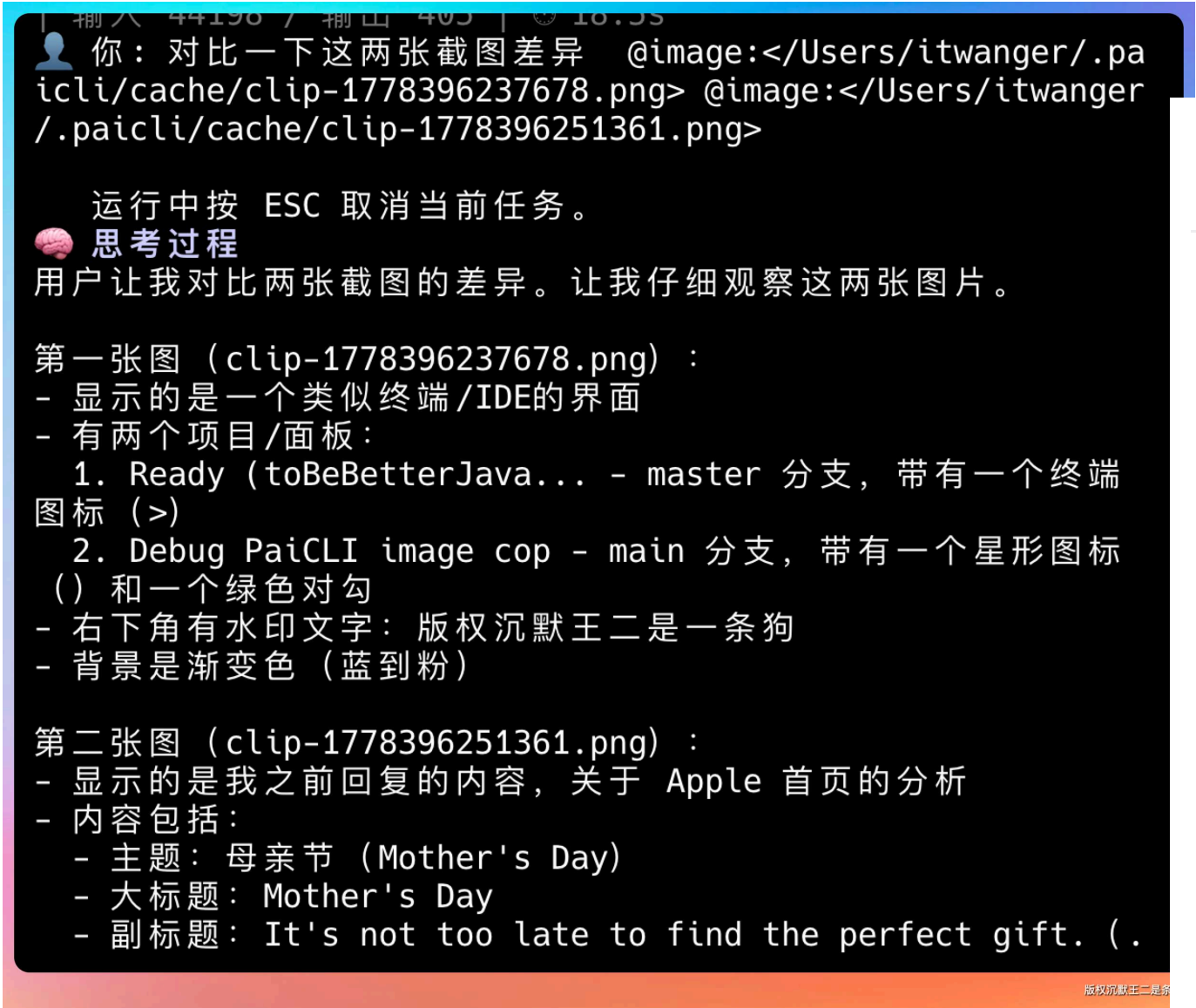
04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上



目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上

实现上，`ImageReferenceParser.findRefs()` 会用正则扫描整条输入，提取所有匹配的 `@image` 引用，存入一个 `List<ImageRef>`，然后逐个加载和预处理。所有成功加载的图片都追加到同一条 user message 的 `contentParts` 里，一条消息可以带多个 image block。

07、图片预处理

PaiCLI 不是拿到图片就直接扔给 API 的。

整个决策树在 `ImageProcessor.process()` 方法里，核心逻辑分三层：

java 复制代码

```
// 1) 字节在 5MB 以内 + 无 alpha → 直通原始字节，不做任何处理
if (!overSize && !hasAlpha) {
    return new ProcessedImage(Base64.getEncoder().encodeToString(bytes), ...);
}

// 2) 有 alpha 但字节在 5MB 以内 → 白底 flatten 后 PNG 输出
if (!overSize && hasAlpha) {
    byte[] flattened = writePng(flattenAlpha(image, width, height));
    // 如果 flatten 后仍然过大，落到第 3 层
}

// 3) 超过 5MB → 等比缩放到 2000x2000，优先 PNG，过大再逐级 JPEG 降质
ResizeSize target = fitWithin(originalWidth, originalHeight, 2000, 2000);
BufferedImage resized = resize(image, target.width(), target.height());
```

为什么要分这三层而不是无脑压缩？

因为每一步都有性能和质量的 tradeoff：

第一层的逻辑覆盖了绝大多数日常截图的场景。macOS 的截图工具生成的 PNG 通常在 500KB~3MB 之间，没有 alpha 通道（即使是 PNG 格式），直接发最好，零 CPU 开销。

第二层的 alpha flatten 处理的是 Sketch、Figma 这类设计工具导出的 PNG。这些图通常带透明通道，如果不处理，模型在分析图片内容的时候，透明区域会被渲染成黑色背景，影响识别准确率。白底 flatten 后视觉效果和用户在屏幕上看到的一致。

第三层才是真正的有损压缩，只有超大图片才会走到这里。2000x2000 的上限选择是基于 token 成本考虑的：按 14x14 的 patch 大小粗略估算，2000x2000 的图片会产生两万级别的视觉 token，已经是一笔不小的上下文开销。

GLM-5V-Turbo 当前官方上下文窗口是 200K，再大的图片收益递减严重，模型的注意力也很难覆盖到每一个细节。

alpha flatten 用的是 Java2D 的 **Graphics2D**：先创建一个 **TYPE_INT_RGB**（无 alpha）的 **BufferedImage**，填充白色背景，再把原图绘制上去。

JPEG 质量降级走的是 **ImageIO** 的 **ImageWriteParam**：

java 复制代码

```
private static final float[] JPEG_QUALITIES = {0.85f, 0.70f, 0.55f, 0.40f, 0.25f};

for (float quality : JPEG_QUALITIES) {
    byte[] candidate = writeJpeg(resized, quality);
    if (estimateBase64Size(candidate.length) <= API_IMAGE_MAX_BASE64_SIZE) {
        return candidate; // 找到第一个达标的质量档位就停
    }
}
```

注意这里用的是 **estimateBase64Size()** 而不是直接比较原始字节长度。

API 传输的是 base64 编码后的字符串，base64 会把每 3 字节变成 4 字符，膨胀比是 4/3。一张 3.75MB 的 JPEG，base64 编码后就是 5MB，刚好卡在上限上。如果不考虑这个膨胀比，按原始字节判断的话，就会出现“客户端觉得没超限但 API 返回 413”的情况。

整个预处理管线的执行时间通常在 50ms~200ms 之间（取决于图片大小和是否需要压缩）。

PaiCLI 如何写到简历上

如果大家在做 Agent 的多模态输入或者图片预处理相关的工作，可以这样写进简历：

- **项目名称**：PaiCLI（终端 AI 编程 Agent）
- **项目简介**：基于 Java 构建的终端原生 AI 编程 Agent，支持多模型接入、MCP 生态、图片多模态输入
- **技术栈**：Java 21、Maven、OpenAI-Compatible API、Chrome DevTools Protocol、MCP 协议
- **核心职责**：
 - 升级 LlmClient.Message 协议，将 content 从 String 扩展为 List<ContentPart>，兼容 text/image_base64/image_url 三种类型
 - 设计并实现 **@image** / **@clipboard** 图片输入协议，支持 **file://**、**绝对路径**、**相对路径**、**尖括号包裹**等多种路径格式
 - 实现 macOS 原生剪贴板图片抓取（AppleScript + osascript），支持 PNG / TIFF 双格式兜底和 sips 格式转换，冷启动 30ms
 - 实现 MCP 工具 image content 到 LLM 图片输入的注入链路，通过 tool message + user image message 的消息序列解决 tool role 不支持 content array 的协议限制

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上



5人已点赞



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

1 条评论



粗心

2026年05月11日 23:33

打卡

1 回复

目录

Agent 终于能看图了！ GLM-5V ...

01、为什么 GLM-5.1 看不了图

02、ContentPart 协议升级

03、@image 引用解析

04、@clipboard 剪贴板抓图

05、MCP 截图

06、对比两张截图

07、图片预处理

PaiCLI 如何写到简历上